

Telecom Customer and Subscription Management System

Project Interview Preparation

Contents

1 Telecom Customer and Subscription Management System	3
1.1 Project Interview Preparation	3
1.2 1. Executive Summary	4
1.2.1 Project purpose	4
1.2.2 Actual architecture	4
1.2.3 Main execution flow	5
1.2.4 What is definitely present	5
1.3 2. Build, Runtime, and Repository Structure	6
1.3.1 Build configuration	6
1.3.2 Runtime scripts and resources	6
1.3.3 Important package map	6
1.4 3. Important File-by-File Analysis	7
1.4.1 src/main/java/com/amdocs/telecom/main/MainApplication.java	7
1.4.2 src/main/java/com/amdocs/telecom/controller/CustomerController.java	7
1.4.3 src/main/java/com/amdocs/telecom/controller/AdminController.java	8
1.4.4 src/main/java/com/amdocs/telecom/util/DBConnection.java	8
1.4.5 src/main/java/com/amdocs/telecom/factory/DAOFactory.java	8
1.4.6 src/main/java/com/amdocs/telecom/factory/ServiceFactory.java	8
1.4.7 Service interfaces and implementations	8
1.4.8 src/main/java/com/amdocs/telecom/service/impl/AuthenticationServiceImpl.java	9
1.4.9 src/main/java/com/amdocs/telecom/service/impl/CustomerServiceImpl.java	9
1.4.10src/main/java/com/amdocs/telecom/service/impl/PlanServiceImpl.java	9
1.4.11src/main/java/com/amdocs/telecom/service/impl/SubscriptionServiceImpl.java	9
1.4.12src/main/java/com/amdocs/telecom/service/impl/BillingServiceImpl.java	10
1.4.13src/main/java/com/amdocs/telecom/service/impl/PaymentServiceImpl.java	10
1.4.14src/main/java/com/amdocs/telecom/service/impl/UsageServiceImpl.java	10
1.4.15src/main/java/com/amdocs/telecom/service/impl/ComplaintServiceImpl.java	10
1.4.16src/main/java/com/amdocs/telecom/service/impl/ReportServiceImpl.java	10
1.4.17DAO interfaces and implementations	10
1.4.18src/main/java/com/amdocs/telecom/strategy/*	11
1.4.19src/main/java/com/amdocs/telecom/scheduler/PaymentNotificationService.java	11
1.4.20src/main/java/com/amdocs/telecom/scheduler/UsageProcessor.java	11
1.4.21src/main/java/com/amdocs/telecom/scheduler/AccountMonitor.java	11
1.4.22src/main/java/com/amdocs/telecom/scheduler/BillingScheduler.java	12
1.4.23src/main/java/com/amdocs/telecom/scheduler/ConsoleActivityMonitor.java	12
1.4.24src/main/java/com/amdocs/telecom/security/PasswordUtil.java	12
1.4.25src/main/java/com/amdocs/telecom/security/CaptchaGenerator.java	12
1.4.26src/main/java/com/amdocs/telecom/security/OTPService.java	12
1.4.27src/main/java/com/amdocs/telecom/validation/ValidationUtil.java	12
1.4.28src/main/java/com/amdocs/telecom/report/ReportGenerator.java	12
1.4.29DTO, model, and enum classes	12
1.5 4. Core Java and OOP Concepts	12
1.5.1 Classes and objects	12
1.5.2 Encapsulation	13
1.5.3 Inheritance	13
1.5.4 Polymorphism	13

1.5.5	Abstraction	13
1.5.6	Interfaces	13
1.5.7	Abstract classes	13
1.5.8	Method overloading	14
1.5.9	Method overriding	14
1.5.10	Constructors	14
1.5.11	Access modifiers	14
1.5.12	static	14
1.5.13	final	14
1.5.14	this	14
1.5.15	super	14
1.5.16	Composition, aggregation, and association	14
1.6	5. Functional Interfaces	15
1.6.1	Definition	15
1.6.2	Functional-interface inventory	15
1.6.3	Predicate<T> in PlanServiceImpl	15
1.6.4	Runnable in notification processing	15
1.6.5	Callable<T> in AccountMonitor	16
1.6.6	Interview answers for absent interfaces	16
1.7	6. Lambda Expressions	16
1.7.1	What a lambda is	16
1.7.2	Verified lambda categories	16
1.7.3	Representative original and equivalent code	16
1.7.4	Scope and effectively final variables	18
1.7.5	Lambda interview chain	18
1.8	7. Stream API	18
1.8.1	Verified operators	18
1.8.2	Representative pipeline: plan filtering	19
1.8.3	Representative pipeline: top customers	19
1.8.4	Representative pipeline: audit activity monitor	19
1.8.5	Stream versus collection	19
1.8.6	Stream versus loop	19
1.8.7	Sequential versus parallel stream	19
1.9	8. Multithreading and Concurrency	19
1.9.1	Overall status	19
1.9.2	PaymentNotificationService: producer-consumer	20
1.9.3	UsageProcessor: four-worker executor	20
1.9.4	AccountMonitor: Callable and Future	20
1.9.5	BillingScheduler: scheduled work	20
1.9.6	ConsoleActivityMonitor: scheduled polling	20
1.9.7	OTPService: concurrent collection	20
1.9.8	Required interview answers	20
1.10	9. Collections	21
1.10.1	Collections found	21
1.10.2	ArrayList versus LinkedList	21
1.10.3	HashMap versus TreeMap	21
1.10.4	Complexity points	21
1.11	10. Exception Handling	22
1.11.1	Exception hierarchy	22
1.11.2	Actual strategy	22
1.11.3	Good examples	22
1.11.4	Weaknesses	22
1.11.5	Interview answer	22
1.12	11. Database and SQL	22
1.12.1	Database type	22
1.12.2	Tables	23
1.12.3	Key relationships	23
1.12.4	Constraints and indexes	23
1.12.5	JDBC flow	23
1.12.6	Prepared statements	23
1.12.7	Transactions	24

1.12.8	Connection handling	24
1.12.9	Currency	24
1.13	12. Design Patterns	24
1.13.1	DAO / Repository-style persistence	24
1.13.2	Service Layer	24
1.13.3	Factory	24
1.13.4	Singleton	24
1.13.5	Strategy	24
1.13.6	Producer-consumer	25
1.13.7	Observer-like polling	25
1.13.8	DTO	25
1.13.9	MVC	25
1.13.10	Dependency Injection	25
1.13.11	Builder, Observer, Repository naming, Adapter, Decorator	25
1.14	13. SOLID Evaluation	25
1.14.1	5 — Single Responsibility	25
1.14.2	0 — Open/Closed	25
1.14.3	L — Liskov Substitution	25
1.14.4	I — Interface Segregation	25
1.14.5	D — Dependency Inversion	26
1.15	14. REST and API Assessment	26
1.15.1	Status	26
1.16	15. Code Quality, Security, and Performance Review	26
1.16.1	Strengths	26
1.16.2	Findings	26
1.16.3	Interview framing	28
1.17	16. Interview Question Bank	28
1.17.1	Level 1 — Basic	28
1.17.2	Level 2 — Intermediate	28
1.17.3	Level 3 — Advanced	29
1.17.4	Level 4 — Tricky/follow-up	29
1.18	17. Interview Question Chains	30
1.18.1	OOB chain	30
1.18.2	Collections chain	30
1.18.3	Lambda/functional-interface chain	30
1.18.4	Stream chain	30
1.18.5	Multithreading chain	30
1.18.6	SQL/database chain	30
1.18.7	Exception chain	31
1.18.8	REST/API chain	31
1.18.9	Design-pattern chain	31
1.18.10	SOLID chain	31
1.19	18. Project Explanation for Interviews	31
1.19.1	30-second answer	31
1.19.2	1-minute answer	31
1.19.3	3-minute answer	32
1.19.4	Discussing contribution	32
1.20	19. “Why Did You Use This?” Answers	32
1.21	20. Final Revision Sheet	33
1.22	21. Final Repository Verification	35

1 Telecom Customer and Subscription Management System

1.1 Project Interview Preparation

Evidence policy: This document is based on the authoritative source under `src/main/java` and `src/main/resources`, plus `pom.xml`, `README.md`, `run.bat`, and the repository tests/artifacts. `bin/` and `target/` are generated or compiled trees and are not treated as independent source implementations. Where a feature is absent, this document says **Not implemented/found in this project**.

Evaluation baseline: The project is a Java 17, interactive console application. It is not a Spring Boot application and it does not expose REST endpoints.

1.2 1. Executive Summary

1.2.1 Project purpose

The project is a Telecom Customer and Subscription Management System, abbreviated in the source as TCSMS. It models the operational workflows of a telecom provider:

- Customer registration, login, password recovery, and account security
- Telecom plan browsing, searching, filtering, and administration
- SIM inventory and mobile-subscription activation
- Plan changes and subscription history
- Usage recording and batch processing
- Monthly bill generation and plan-change billing
- Simulated UPI, card, and net-banking payments
- Complaints, resolution, notifications, and audit logging
- Revenue, customer, complaint, and usage-related reports
- Background notification, billing, overdue-account, usage, and console-activity processing

The application persists data with JDBC in a relational database. It attempts MySQL first and falls back to an embedded file-backed H2 database when MySQL is unavailable.

1.2.2 Actual architecture

The project is closest to a layered console application:

```
\NormalTok{User at console}
\NormalTok{{}}
\NormalTok{{v}}
\NormalTok{MainApplication}
\NormalTok{{}}
\NormalTok{{+{-}{-}}\textgreater{} CustomerController / AdminController}
\NormalTok{{}}
\NormalTok{{v}}
\NormalTok{{Service interfaces}}
\NormalTok{{}}
\NormalTok{{v}}
\NormalTok{{Service implementations}}
\NormalTok{{}}
\NormalTok{{+{-}{-}}\textgreater{} validation, security, business rules}
\NormalTok{{+{-}{-}}\textgreater{} PaymentStrategy implementations}
\NormalTok{{+{-}{-}}\textgreater{} ReportGenerator}
\NormalTok{{}}
\NormalTok{{v}}
\NormalTok{{DAO interfaces}}
\NormalTok{{}}
\NormalTok{{v}}
\NormalTok{{JDBC DAO implementations}}
\NormalTok{{}}
\NormalTok{{v}}
\NormalTok{{DBConnection / DriverManager}}
\NormalTok{{}}
\NormalTok{{v}}
\NormalTok{{MySQL or file{-}backed H2 database}}
```

Background paths run beside the interactive console:

```
\NormalTok{MainApplication}
\NormalTok{{}}
\NormalTok{{+{-}{-}}\textgreater{} PaymentNotificationService}
\NormalTok{{}}
\NormalTok{{+{-}{-}}\textgreater{} bounded}
↪ BlockingQueue\textless{}Notification\textgreater{}
\NormalTok{{+{-}{-}}\textgreater{} three worker threads}
\NormalTok{{+{-}{-}}\textgreater{} AuditAndNotificationDAOImpl}
```

Other scheduler/worker classes are available for administrator workflows. `BillingScheduler.start(...)` exists, but `MainApplication` does not automatically start it. The administrator menu can trigger billing manually through `BillingScheduler.triggerNow()`.

1.2.3 Main execution flow

1. `MainApplication.main` resets verbose JUL logging.
2. `DBConnection.getInstance()` initializes the database manager.
3. Static initialization in `DBConnection` loads `db.properties` and attempts database initialization.
4. `schema.sql`, the administrator-column compatibility check, and `seed.sql` are executed.
5. `MainApplication` starts `PaymentNotificationService`.
6. The main console menu creates either `CustomerController` or `AdminController`.
7. Controllers read console input, call service implementations, and print results.
8. Services apply validation and business rules, then call DAO interfaces.
9. DAO implementations obtain JDBC connections, execute SQL, map `ResultSet` rows to model objects, and close JDBC resources with try-with-resources.
10. On exit, notification workers are shut down and the Scanner is closed.

1.2.4 What is definitely present

Area	Status	Evidence
Java 17	Used	<code>pom.xml</code> , compiler release 17
Console UI	Used	<code>MainApplication</code> , <code>CustomerController</code> , <code>AdminController</code>
JDBC	Used	<code>DBConnection</code> , classes under <code>dao.impl</code>
MySQL/H2	Used	<code>db.properties</code> , <code>DBConnection</code> , Maven dependencies
DAO pattern	Used	DAO interfaces plus <code>dao.impl</code> classes
Service layer	Used	service interfaces plus <code>service.impl</code> classes
Factory pattern	Used	<code>DAOFactory</code> , <code>ServiceFactory</code> , <code>PaymentStrategyFactory</code>
Strategy pattern	Used	<code>PaymentStrategy</code> , UPI/card/net-banking strategies
Lambda expressions	Used	streams and executor tasks
Stream API	Used	<code>plan</code> , <code>report</code> , <code>controller</code> , <code>monitor</code> , subscription code
Functional interfaces	Partially used	<code>Predicate</code> , <code>Runnable</code> , <code>Callable</code> ; no direct <code>Function/Consumer</code> declarations
Multithreading	Used	notification, usage, account, billing, console monitors
REST API	Not implemented/found in this project	no web framework, route annotations, or HTTP controllers
Automated tests	Not implemented/found in this project	no test source files or test dependency found
Abstract classes	Not implemented/found in this project	no source class extends an application abstract class

Area	Status	Evidence
Dependency injection framework	Not implemented/found in this project	objects are commonly created with new

1.3 2. Build, Runtime, and Repository Structure

1.3.1 Build configuration

File: pom.xml

Purpose: Maven project metadata, Java version, dependencies, and plugins.

Coordinates:

- Group: com.amdocs.telecom
- Artifact: telecom-customer-subscription-management
- Version: 1.0.0
- Packaging: jar

Dependencies:

Dependency	Version	Actual use
com.h2database:h2	2.2.224	Embedded/file-backed fallback database
com.mysql:mysql-connector-j	8.3.0	MySQL JDBC driver
org.mindrot:jbcrypt	0.4	BCrypt password hashing

Plugins:

- maven-compiler-plugin:3.11.0, configured with Java release 17
- exec-maven-plugin:3.1.0, configured for com.amdocs.telecom.main.MainApplication

There are no JUnit, TestNG, Mockito, Spring, Jakarta REST, servlet, JSON, connection-pool, or web-server dependencies in pom.xml.

1.3.2 Runtime scripts and resources

- run.bat: compiles source directly with javac, downloads/uses jars under lib, copies resources, and launches the application. This is the practical non-Maven launch path.
- view-db.bat: database viewing helper.
- src/main/resources/db.properties: database configuration.
- src/main/resources/schema.sql: table and index definitions.
- src/main/resources/seed.sql: demo data.
- data/: runtime H2 database location.
- output/: generated reports/invoices.
- reports/: evaluation notes and sample invoice/report files.
- bin/: compiled artifact tree; not authoritative source.
- target/: Maven/compile output; not authoritative source.
- sources.txt: generated source-file list used by the batch build process.

1.3.3 Important package map

Package	Responsibility
controller	Console menus, input, presentation, workflow orchestration
dao	Persistence contracts
dao.impl	JDBC persistence implementations
dto	Input-transfer object for registration

Package	Responsibility
exception	Application exception hierarchy
factory	DAO and service construction
main	Application entry point
model	Domain objects and enums
report	CSV, text, HTML, and PDF-style output generation
scheduler	Background and scheduled processing
security	Password, CAPTCHA, OTP utilities
service	Business-service contracts
service.impl	Business rules and use-case implementations
strategy	Payment strategy abstraction and implementations
util	Database connection management
validation	Reusable validation rules

1.4 3. Important File-by-File Analysis

1.4.1 src/main/java/com/amdocs/telecom/main/MainApplication.java

Purpose: Application entry point and top-level console lifecycle.

Important class: MainApplication

Important methods:

- `main(String[] args)`: initializes the database, starts notifications, displays the main menu, dispatches to customer/admin controllers, and shuts down resources.
- `printMainMenu()`: prints the three top-level options.
- `readInt(Scanner scanner)`: parses an integer and returns -1 on any exception.
- `getNotificationService()`: exposes the static notification service.

Concepts: entry point, static, switch arrow syntax, object creation, exception handling, lifecycle management.

Relationships: constructs CustomerController, AdminController, and PaymentNotificationService; accesses DBConnection.

Interview point: The main class is a composition root only in a limited sense. It wires some objects manually, but there is no framework-managed dependency injection.

1.4.2 src/main/java/com/amdocs/telecom/controller/CustomerController.java

Purpose: Interactive customer portal.

Important methods: `showLoginMenu`, `handleLogin`, `handleRegistration`, `handleForgotPassword`, `showCustomerDashboard`, `browsePlans`, `searchPlans`, `filterPlansByPrice`, `viewSubscriptions`, `subscribeToPlan`, `changePlan`, `viewBills`, `makePayment`, `executePaymentWithMethod`, `viewUsage`, `raiseComplaint`, `trackCustomerComplaints`, and `viewNotifications`.

Responsibilities actually combined in this class:

- Console presentation and input parsing
- Validation prompts
- Login and registration workflow
- Service construction and orchestration
- Payment input and payment strategy selection
- Invoice generation
- Stream-based selection and display
- Customer authorization decisions

This is useful to discuss as an example of a controller that has become too large. A stronger design would move input/presentation helpers, authorization, payment orchestration, and report

generation into separate collaborators.

1.4.3 src/main/java/com/amdocs/telecom/controller/AdminController.java

Purpose: Interactive administrator portal.

Important methods: showLoginMenu, handleAdminLogin, showAdminDashboard, viewAllPlans, addNewPlan, togglePlanStatus, viewAllCustomers, viewAllSubscriptions, viewAllPayments, reactivateSubscription, viewSimInventory, generateBillingCycle, viewUnpaidBills, scanOverdueAccounts, processBulkUsage, showRevenueReports, showCustomerDistribution, manageComplaints, handleResolveComplaint, showComplaintHotspotsAndAnalytics, viewAuditLogs, handleAdminForgotPassword, handleAdminChangePassword, and toggleLive-ActivityMonitor.

Important observation: The administrator menu is the main integration surface for reports, billing, usage processing, overdue-account processing, complaint management, and live audit activity.

1.4.4 src/main/java/com/amdocs/telecom/util/DBConnection.java

Purpose: Singleton database manager and schema/seed initializer.

Important members/methods:

- private static DBConnection instance
- private static Properties properties
- private static boolean useFallbackH2
- getInstance(): synchronized lazy singleton accessor.
- loadProperties(): loads db.properties from the classpath.
- getConnection(): tries MySQL, then permanently switches to H2 after failure.
- getActiveDatabaseName(): reads database metadata.
- testAndInitializeDatabase(): runs schema and seed initialization.
- runScript(...): reads semicolon-terminated SQL statements and executes them.
- ensureAdministratorAccountStatusColumn(...): performs a compatibility migration check.

Important design detail: This class is called a connection manager, but it does not hold one shared JDBC connection and it is not a connection pool. Each DAO calls getConnection() and receives a separate DriverManager connection.

Potential issue: runScript is a simple line-based SQL script runner. It is adequate for the current scripts but is not a general SQL parser.

1.4.5 src/main/java/com/amdocs/telecom/factory/DAOFactory.java

Purpose: Centralizes construction of DAO implementation objects.

Pattern: Factory.

Interview explanation: It hides the concrete JDBC class from callers that choose to use the factory. However, the project does not consistently route all construction through this factory.

1.4.6 src/main/java/com/amdocs/telecom/factory/ServiceFactory.java

Purpose: Centralizes construction of service implementation objects.

Pattern: Factory.

Limitation: Controllers and services still instantiate concrete services or DAOs directly in multiple places, so the factory is only partially used as a dependency boundary.

1.4.7 Service interfaces and implementations

Contract	Implementation	Main responsibility
AuthenticationService	AuthenticationServiceImpl	Login, failed attempts, logout, login history
CustomerService	CustomerServiceImpl	Registration, customer lookup, password changes

Contract	Implementation	Main responsibility
PlanService	PlanServiceImpl	Plan queries and stream-based filtering
SubscriptionService	SubscriptionServiceImpl	Subscribe, change plan, activate/deactivate
BillingService	BillingServiceImpl	Monthly and plan-change bills
PaymentService	PaymentServiceImpl	Payment authorization, transaction, audit, notification
UsageService	UsageServiceImpl	Usage validation, persistence, summaries
ComplaintService	ComplaintServiceImpl	Complaint creation, lookup, resolution, analytics
ReportService	ReportServiceImpl	Revenue, customer, and plan reports

These interfaces are examples of abstraction and polymorphism. A caller can depend on `PaymentService` rather than `PaymentServiceImpl`, even though the current construction style often creates the implementation directly.

1.4.8 `src/main/java/com/amdocs/telecom/service/impl/AuthenticationServiceImpl.java`

Important method: `login(String username, String password, String expectedCaptcha, String inputCaptcha)`.

Observed flow:

1. Compares CAPTCHA values.
2. Looks up the customer.
3. Rejects invalid credentials.
4. Checks account status and failed-attempt state.
5. Uses password verification through `PasswordUtil`.
6. Records login history and updates failed attempts as appropriate.
7. Throws `AuthenticationException` for authentication failures/database errors.

Security discussion: The project has CAPTCHA and logout logic, but checking whether a username exists before password verification can reveal account existence.

1.4.9 `src/main/java/com/amdocs/telecom/service/impl/CustomerServiceImpl.java`

Important method: `registerCustomer(CustomerRegistrationDTO dto)`.

Observed flow: validates names, age, email, mobile number, and password complexity; checks username/email/mobile uniqueness; hashes the password; constructs a `Customer`; and persists it through `CustomerDAO`.

Pattern: DTO plus service-layer validation.

1.4.10 `src/main/java/com/amdocs/telecom/service/impl/PlanServiceImpl.java`

Important methods: `searchPlansByName`, `filterPlansByPrice`, `filterPlansByData`, and `sortPlansByPrice`.

Functional-interface use: declares `Predicate<TelecomPlan>` variables and passes lambdas to `Stream.filter`.

1.4.11 `src/main/java/com/amdocs/telecom/service/impl/SubscriptionServiceImpl.java`

Important methods: `subscribeToPlan` and `changePlan`.

`subscribeToPlan` checks that the plan exists and is active, prevents a duplicate active plan, finds an available SIM, saves the subscription, and attempts to generate the initial bill. It explicitly reports the partial-failure case where the subscription is created but bill generation fails.

`changePlan` delegates transactional plan/history updates to `SubscriptionDAOImpl.changePlan`, then the controller/service flow generates the related bill separately.

Transaction issue: subscription creation and initial billing do not share one transac-

tion/connection.

1.4.12 src/main/java/com/amdocs/telecom/service/impl/BillingServiceImpl.java

Important methods: generateMonthlyBill and generatePlanChangeBill.

The service checks for existing bills, loads the subscription and plan, calculates rental/usage/tax/discount/total builds a Bill, and saves it through BillingDAO.

Data-quality issue: money is represented mainly with double, and due-date logic uses a fixed date offset rather than deriving a due date from the billing period.

1.4.13 src/main/java/com/amdocs/telecom/service/impl/PaymentServiceImpl.java

Important method: processPayment(int billId, int customerId, double amount, String paymentModeStr, String processedBy).

Observed flow:

1. Opens a JDBC connection.
2. Disables auto-commit.
3. Loads and locks the bill with FOR UPDATE.
4. Confirms that the bill belongs to the requesting customer.
5. Rejects already-paid bills.
6. Requires the amount to exactly match the total.
7. Converts the selected mode to PaymentMode.
8. Obtains a concrete PaymentStrategy from PaymentStrategyFactory.
9. Persists the payment using the same connection.
10. Marks the bill paid.
11. Writes an audit record.
12. Commits on success, rolls back on failure.
13. Queues an asynchronous notification after successful processing.

This is one of the strongest interview examples in the project because it demonstrates authorization, a database transaction, row locking, a factory, strategy polymorphism, and asynchronous notification.

1.4.14 src/main/java/com/amdocs/telecom/service/impl/UsageServiceImpl.java

Important method: recordUsage(...).

Validates positive quantity and a nonempty unit, creates a usage record, persists it, and attempts audit logging. A SQL error is converted to a null result in the current implementation, which is a poor failure contract because callers must distinguish a real absence from an error.

1.4.15 src/main/java/com/amdocs/telecom/service/impl/ComplaintServiceImpl.java

Important methods: lodgeComplaint, lookup methods, and resolveComplaint.

It converts a string category into ComplaintCategory, validates description, persists the complaint, and updates status/resolution. The constructor can create sample complaints if none exist, which is surprising for a production service and should be separated into seed/test setup.

1.4.16 src/main/java/com/amdocs/telecom/service/impl/ReportServiceImpl.java

Important methods: getHighestConsumingCustomers, getCustomerDistributionByCity, getRevenueSummaryByPlan, and getAverageMonthlyRevenuePerCustomer.

Uses stream grouping and aggregation. Two naming/semantic issues should be discussed candidly:

- getHighestConsumingCustomers ranks by bill totals, not actual usage quantities.
- getAverageMonthlyRevenuePerCustomer divides total paid revenue by total customer count, not necessarily by paying customers or by the number of months.

1.4.17 DAO interfaces and implementations

The DAO interfaces define persistence contracts and the dao.impl classes contain JDBC code. Main pairs are:

Interface	Implementation	Representative operations
CustomerDAO	CustomerDAOImpl	save, find by id/username/email, update password, failed login count
AdminDAO	AdminDAOImpl	find admin, update password/account status
PlanDAO	PlanDAOImpl	save, find active/all, update status
SubscriptionDAO	SubscriptionDAOImpl	save, find by customer, find SIMs, status update, plan change
BillingDAO	BillingDAOImpl	save, find by customer, unpaid bills, lock bill, update status
PaymentDAO	PaymentDAOImpl	save, find by customer/bill, list all
UsageDAO	UsageDAOImpl	save, batch save, find usage, summaries, charges
ComplaintDAO	ComplaintDAOImpl	save, lookup, resolution, city/category analytics
AuditAndNotificationDAO	AuditAndNotificationDAOImpl	audit logs, notifications, mark/read operations

Most dynamic values use PreparedStatement. Some constant, non-interpolated list and analytics queries use raw Statement, including findAll methods and complaint/audit aggregations. This is not currently direct injection because user input is not interpolated into those constant strings, but standardizing on PreparedStatement would make the policy consistent.

1.4.18 src/main/java/com/amdocs/telecom/strategy/*

- PaymentStrategy: abstraction with validateAndProcess(double amount, String customerAccount).
- UpiPaymentStrategy, CardPaymentStrategy, NetBankingPaymentStrategy: concrete implementations.
- PaymentStrategyFactory: maps PaymentMode to a strategy.

The strategies simulate validation and processing. They do not call a real payment gateway.

1.4.19 src/main/java/com/amdocs/telecom/scheduler/PaymentNotificationService.java

Uses a bounded LinkedBlockingQueue<Notification> of capacity 100 and Executors.newFixedThreadPool(). start() submits three Runnable lambdas. sendNotification() is the producer and uses nonblocking offer(). processNotification() persists the notification through AuditAndNotificationDAO.

Trade-off: offer() means a full queue drops a notification rather than applying backpressure or retrying. Shutdown stops workers but does not explicitly drain all queued notifications.

1.4.20 src/main/java/com/amdocs/telecom/scheduler/UsageProcessor.java

Uses a four-worker executor to process batches and performs batch JDBC inserts. A synchronized lock protects the shared processed counter. This is a direct example of coordination around shared mutable state.

1.4.21 src/main/java/com/amdocs/telecom/scheduler/AccountMonitor.java

Creates two Callable tasks and retrieves their results with Future.get and timeouts. It scans overdue bills and marks accounts or performs suspension-related work.

Concurrency concern: scanning and updating the same unpaid-bill data concurrently can produce race conditions or inconsistent observations unless the database operations provide the required isolation/locking.

1.4.22 src/main/java/com/amdocs/telecom/scheduler/BillingScheduler.java

Defines a Runnable billing task and schedules it with `ScheduledExecutorService.scheduleAtFixedRate`. It is implemented but not started automatically from `MainApplication`.

1.4.23 src/main/java/com/amdocs/telecom/scheduler/ConsoleActivityMonitor.java

Uses a single-thread scheduled executor to poll audit logs every two seconds. `start`, `toggle`, and `stop` are synchronized, and `lastAuditId`/`running` are volatile.

1.4.24 src/main/java/com/amdocs/telecom/security/PasswordUtil.java

Uses `JBCrypt` for hashing and verification. This is preferable to storing plaintext passwords. The surrounding application still contains seeded/demo credentials, which are appropriate for a case study but not for production deployment.

1.4.25 src/main/java/com/amdocs/telecom/security/CaptchaGenerator.java

Generates CAPTCHA values for the console login workflow.

1.4.26 src/main/java/com/amdocs/telecom/security/OTPService.java

Stores OTP entries in a static `ConcurrentHashMap`. This gives thread-safe map operations within one JVM, but OTP state is not shared across multiple application instances and rate limiting/durable audit are not present.

1.4.27 src/main/java/com/amdocs/telecom/validation/ValidationUtil.java

Centralizes reusable validation checks such as email, mobile, password, and input formats.

1.4.28 src/main/java/com/amdocs/telecom/report/ReportGenerator.java

Generates customer/revenue CSV output, text invoices, HTML invoices, and PDF-style output. It uses `try-with-resources` for file writers/streams.

Security concern: HTML output inserts customer/billing values without an explicit HTML escaping layer. CSV escaping is also selective rather than a fully centralized CSV library/encoder.

1.4.29 DTO, model, and enum classes

The model package contains mutable domain objects with fields, constructors, getters, and setters. Important models include:

- `Customer`, `Administrator`: identities and account data
- `TelecomPlan`: tariff-plan data
- `SIMCard`: SIM inventory
- `MobileSubscription`, `SubscriptionHistory`: subscription state and changes
- `UsageRecord`: usage quantity, unit, and charge
- `Bill`: billing values and status
- `Payment`: payment record
- `Complaint`, `ComplaintCategory`: complaint workflow
- `Notification`, `AuditLog`, `LoginHistory`: operational/security records

Enums include `PaymentMode`, `SimType`, `SubscriptionType`, `UsageType`, and `ComplaintCategory`. Enums restrict values more safely than arbitrary strings, although some database/service APIs still accept strings and convert them.

`CustomerRegistrationDTO` carries registration input separately from the persisted `Customer` model.

1.5 4. Core Java and OOP Concepts**1.5.1 Classes and objects**

Definition: A class defines state and behavior; an object is a runtime instance of that class.

Project evidence: `MainApplication` creates new `CustomerController(scanner)`, new `AdminController(scanner)`, and new `PaymentNotificationService()`. DAOs create model objects while mapping JDBC rows.

Why used: The project models real entities and services as objects with state and behavior.

If removed: There would be no domain or service objects; the application would need procedural code and would lose the current separation.

Interview answer: “My project uses classes for domain objects such as Customer, Bill, and MobileSubscription, and for services such as PaymentServiceImpl. Runtime objects carry the current customer, bill, connection, or worker-pool state.”

1.5.2 Encapsulation

Definition: Keep fields private and expose controlled methods.

Evidence: Model objects use private fields with getters/setters; DBConnection hides its constructor and connection-selection logic; PaymentNotificationService keeps its queue and executor private.

Why used: Callers should not mutate connection state or worker infrastructure directly.

Limitation: Mutable setters allow broad state changes, and controllers access many service details. Encapsulation is present but not immutable-domain-model based.

1.5.3 Inheritance

Definition: A class derives from another class.

Used: The custom exception hierarchy uses inheritance:

```
\KeywordTok{public} \KeywordTok{class} \NormalTok{ TelecomException } \KeywordTok{extends}
↪ \BuiltInTok{Exception} \OperatorTok{\{\} } \OperatorTok{\{\} }
\KeywordTok{public} \KeywordTok{class} \BuiltInTok{ValidationException}
↪ \KeywordTok{extends} \NormalTok{ TelecomException } \OperatorTok{\{\} } \OperatorTok{\{\} }
\KeywordTok{public} \KeywordTok{class} \BuiltInTok{AuthenticationException}
↪ \KeywordTok{extends} \NormalTok{ TelecomException } \OperatorTok{\{\} } \OperatorTok{\{\} }
```

Not used: No domain model inheritance hierarchy or abstract base class was found.

1.5.4 Polymorphism

Definition: Code uses a common contract while runtime objects provide different implementations.

Evidence: PaymentStrategy strategy = PaymentStrategyFactory.getStrategy(mode); can refer to UpiPaymentStrategy, CardPaymentStrategy, or NetBankingPaymentStrategy. DAO and service interfaces similarly allow implementations to be substituted.

Why used: Payment behavior varies by mode without putting all payment-specific logic in PaymentServiceImpl.

If removed: PaymentServiceImpl would need a large conditional block and would be harder to extend.

1.5.5 Abstraction

Definition: Expose what a component does while hiding how it does it.

Evidence: PaymentStrategy, DAO interfaces, and service interfaces hide implementation details.

Partial status: The abstraction exists, but dependency construction often bypasses it by directly creating concrete implementations.

1.5.6 Interfaces

Used: Nine service interfaces, nine DAO interfaces, and PaymentStrategy are present.

Why: Contracts support substitution and separate callers from JDBC/concrete strategy details.

Interview caution: “Having interfaces” alone does not prove dependency inversion is fully achieved. In this project, direct new construction creates coupling despite the interfaces.

1.5.7 Abstract classes

Status: Not implemented/found in this project. No application abstract class was found.

Interview preparation: Explain that an abstract class can share state and implemented behavior,

while an interface defines a contract and, in modern Java, may also contain default/static methods. In this project, interfaces are used instead of abstract base classes.

1.5.8 Method overloading

Used: `AuditAndNotificationDAO.logAudit(AuditLog)` and `logAudit(Connection, AuditLog)` are overloaded. DAO methods such as `BillingDAO.findById(int)` and `findById(Connection, int)` also provide context-specific overloads.

Why: The overload that receives a connection allows a caller to participate in an existing transaction.

1.5.9 Method overriding

Used: Every `*DAOImpl` class overrides methods declared by its DAO interface, and every `*ServiceImpl` overrides methods declared by its service interface. Payment strategy classes override `validateAndProcess`.

1.5.10 Constructors

Constructors initialize objects such as `PaymentNotificationService`, which creates its queue, executor, and DAO. `DBConnection` has a private constructor to enforce singleton construction. `CustomerRegistrationDTO` and model classes use constructors/setters to hold input/state.

1.5.11 Access modifiers

- **private:** internal state such as `DBConnection.instance`, queues, and service dependencies.
- **public:** application entry points, service methods, DAO contracts, and model accessors.
- **static:** singleton state, utility methods, and the main entry point.
- **final:** dependencies and constants that should not be reassigned.

1.5.12 static

Used: `MainApplication.main`, `DBConnection` singleton state, `PasswordUtil`, `OTPService.otpStorage`, and factory methods.

Risk: Static mutable state, especially OTP storage and singleton fallback state, complicates testing and multi-instance deployment.

1.5.13 final

Used for constants and dependencies such as `private final BlockingQueue<Notification> notificationQueue`. A final reference cannot be reassigned, although the referenced queue remains mutable.

1.5.14 this

Used in constructors and instance methods to distinguish fields from parameters and to pass the current instance, including `this::poll` in `ConsoleActivityMonitor`.

1.5.15 super

Status: No meaningful direct super usage was found in application code. The exception subclasses inherit from parent exceptions but do not need explicit constructor delegation in the simple source form.

1.5.16 Composition, aggregation, and association

- **Composition:** `PaymentNotificationService` owns its queue and worker pool lifecycle. `MainApplication` owns the top-level notification service lifecycle.
- **Aggregation:** A customer has subscriptions, bills, complaints, and notifications represented through database relationships and retrieved collections. The objects are not necessarily constructed as one nested aggregate in memory.
- **Association:** `MobileSubscription` references customer/plan/SIM identifiers; `Payment` references bill/customer identifiers; DAO/service objects collaborate through fields and method calls.

1.6 5. Functional Interfaces

1.6.1 Definition

A functional interface has exactly one abstract method, called a Single Abstract Method (SAM). A lambda can implement that contract. `@FunctionalInterface` is a compiler check and documentation marker, but it is optional.

1.6.2 Functional-interface inventory

Type	Directly used?	Evidence
<code>Predicate<T></code>	Yes	<code>PlanServiceImpl</code> filtering predicates
<code>Runnable</code>	Yes	notification, usage, billing tasks
<code>Callable<T></code>	Yes	<code>AccountMonitor</code> tasks returning results
<code>Function<T,R></code>	No direct declaration found	Not implemented/found as a named project type
<code>Consumer<T></code>	No direct declaration found	<code>forEach</code> accepts a consumer, but no named <code>Consumer</code> variable was found
<code>Supplier<T></code>	Not directly declared	Not implemented/found as a named project type
<code>BiFunction<T,U,R></code>	Not implemented/found	No direct use found
<code>BiConsumer<T,U></code>	Not implemented/found	No direct use found
<code>UnaryOperator<T></code>	Not implemented/found	No direct use found
<code>BinaryOperator<T></code>	Not implemented/found	No direct use found
Custom <code>@FunctionalInterface</code>	Not implemented/found	No custom annotated interface found

1.6.3 Predicate<T> in PlanServiceImpl

```
\BuiltInTok{Predicate}\OperatorTok{\textless{}}\NormalTok{TelecomPlan}\OperatorTok{\textg
↪ reater{}}\NormalTok{ nameContains }\OperatorTok{=}
\NormalTok{ p }\OperatorTok{{-}\textgreater{}}\NormalTok{
↪ p}\OperatorTok{.}\FunctionTok{getPlanName}\OperatorTok{().}\FunctionTok{toLowerCase}\
↪ OperatorTok{().}\FunctionTok{contains}\OperatorTok{({}\NormalTok{keyword}\OperatorTok{
↪ .}\FunctionTok{toLowerCase}\OperatorTok{()});}

\ControlFlowTok{return}\NormalTok{
↪ plans}\OperatorTok{.}\FunctionTok{stream}\OperatorTok{()}
  \OperatorTok{.}\FunctionTok{filter}\OperatorTok{({}\NormalTok{nameContains}\Operator
  ↪ orTok{)}}
  \OperatorTok{.}\FunctionTok{collect}\OperatorTok{({}\NormalTok{Collectors}\Operator
  ↪ rTok{.}\FunctionTok{toList}\OperatorTok{()});}
```

The predicate accepts a plan and returns a boolean. It is reusable as a named condition and is passed to filter.

1.6.4 Runnable in notification processing

```
\NormalTok{workerPool}\OperatorTok{.}\FunctionTok{submit}\OperatorTok{({}\BuiltInTok{Runn
↪ able}\OperatorTok{()}\OperatorTok{{-}\textgreater{}}\OperatorTok{\}
\ControlFlowTok{while}\OperatorTok{({}\NormalTok{running}\OperatorTok{)}}
↪ \OperatorTok{\}
  \BuiltInTok{Notification}\NormalTok{ notif }\OperatorTok{=}\NormalTok{
  ↪ notificationQueue}\OperatorTok{.}\FunctionTok{poll}\OperatorTok{({}\DecValTok{
  ↪ 2}\OperatorTok{,}
  ↪ \BuiltInTok{TimeUnit}\OperatorTok{.}\FunctionTok{SECONDS}\OperatorTok{()});}
```

```

\ControlFlowTok{if} \OperatorTok{({}\NormalTok{notif }}\OperatorTok{!=}
  ↪ \KeywordTok{null}\OperatorTok{)}} \OperatorTok{{}}
    \FunctionTok{processNotification}\OperatorTok{({}\NormalTok{notif }}\OperatorTok{
      ↪ {,}\NormalTok{ workerId}\OperatorTok{)}};
\OperatorTok{}}
\OperatorTok{{}}
\OperatorTok{}};

```

Runnable represents work with no return value. The executor runs the lambda on a worker thread.

1.6.5 Callable<T> in AccountMonitor

```

\BuiltInTok{Callable}\OperatorTok{\textless{}}\BuiltInTok{List}\OperatorTok{\textless{}}\
  ↪ NormalTok{Bill}\OperatorTok{\textgreater{}}\textgreater{}}\NormalTok{ overdueScanTask
  ↪ }\OperatorTok{=}\OperatorTok{({}} \OperatorTok{{-}}\textgreater{}} \OperatorTok{{}}
  ↪ \KeywordTok{...} \OperatorTok{}};
\BuiltInTok{Callable}\OperatorTok{\textless{}}\BuiltInTok{Integer}\OperatorTok{\textgtr
  ↪ er{}}\NormalTok{ markOverdueTask }\OperatorTok{=}\OperatorTok{({}}
  ↪ \OperatorTok{{-}}\textgreater{}} \OperatorTok{{}} \KeywordTok{...} \OperatorTok{}};
\BuiltInTok{Future}\OperatorTok{\textless{}}\BuiltInTok{List}\OperatorTok{\textless{}}\No
  ↪ rmalTok{Bill}\OperatorTok{\textgreater{}}\textgreater{}}\NormalTok{ overdueFuture
  ↪ }\OperatorTok{=}\NormalTok{ executor}\OperatorTok{.}\FunctionTok{submit}\OperatorTok{
  ↪ (}\NormalTok{overdueScanTask}\OperatorTok{)}};

```

Callable<T> differs from Runnable because it returns a value and can throw checked exceptions. Future.get(...) retrieves that result with a timeout.

1.6.6 Interview answers for absent interfaces

Question: "Where did you use Function?"

Answer: "I did not directly declare a Function in this repository. I used Predicate<TelecomPlan> and executor functional interfaces such as Runnable and Callable. Function<T,R> would be appropriate if I extracted a reusable object-to-value transformation from one of the stream pipelines."

Question: "Is forEach a functional interface?"

Answer: "forEach is a method that accepts a Consumer. The project passes lambdas directly to forEach, but I did not find a named Consumer declaration."

1.7 6. Lambda Expressions

1.7.1 What a lambda is

A lambda is a concise implementation of a functional interface. Its general form is (parameters) -> expression or (parameters) -> { statements; }.

1.7.2 Verified lambda categories

1. Stream predicates: p -> ..., bill -> ..., sub -> ...
2. Stream mappings/aggregations: a -> ..., c1, c2 -> ...
3. Executor tasks: () -> { ... }
4. Method references: AuditLog::getAuditId, this::poll
5. Method calls inside forEach: u -> System.out.println(...)

1.7.3 Representative original and equivalent code

Original in PlanServiceImpl:

```

\ControlFlowTok{return}\NormalTok{
  ↪ plans}\OperatorTok{.}\FunctionTok{stream}\OperatorTok{({}}
    \OperatorTok{.}\FunctionTok{filter}\OperatorTok{({}\NormalTok{p
      ↪ }\OperatorTok{{-}}\textgreater{}}\NormalTok{
      ↪ p}\OperatorTok{.}\FunctionTok{getMonthlyRental}\OperatorTok{({}}
      ↪ \OperatorTok{\textless{}}=}\NormalTok{ maxPrice}\OperatorTok{)}}
\OperatorTok{.}\FunctionTok{collect}\OperatorTok{({}\NormalTok{Collectors}\OperatorTok{
  ↪ rTok{.}\FunctionTok{toList}\OperatorTok{({}});

```

Equivalent without a lambda/stream:

```

\BuiltInTok{List}\OperatorTok{\textless{}}\NormalTok{TelecomPlan}\OperatorTok{\textgreater{}}
↪ r{}\NormalTok{ result }\OperatorTok{=}\KeywordTok{new}
↪ \BuiltInTok{ArrayList}\OperatorTok{\textless{}}\textgreater{}}();}
\ControlFlowTok{for} \OperatorTok{({}\NormalTok{TelecomPlan plan
↪ }\OperatorTok{:}\NormalTok{ plans}\OperatorTok{)} }\OperatorTok{\{
  \ControlFlowTok{if} \OperatorTok{({}\NormalTok{plan}\OperatorTok{.}\FunctionTok{getMon
    ↪ thlyRental}\OperatorTok{()}\OperatorTok{\textless{}}=}\NormalTok{
    ↪ maxPrice}\OperatorTok{)} }\OperatorTok{\{
\NormalTok{
↪ result}\OperatorTok{.}\FunctionTok{add}\OperatorTok{({}\NormalTok{plan}\OperatorTok{)} }\OperatorTok{\{
  \OperatorTok{\}}
\OperatorTok{\}}
\ControlFlowTok{return}\NormalTok{ result}\OperatorTok{;}

```

Why the lambda is useful: It keeps the filtering condition adjacent to the pipeline and avoids manual result-list management. The traditional loop may be easier to debug line-by-line and may be clearer for complex multi-step control flow.

Original in AccountMonitor:

```

\BuiltInTok{Callable}\OperatorTok{\textless{}}\BuiltInTok{List}\OperatorTok{\textless{}}\
↪ NormalTok{Bill}\OperatorTok{\textgreater{}}\textgreater{}}\NormalTok{ overdueScanTask
↪ }\OperatorTok{=}\OperatorTok{()}\OperatorTok{-}\textgreater{}} \OperatorTok{\{
  \BuiltInTok{List}\OperatorTok{\textless{}}\NormalTok{Bill}\OperatorTok{\textgreater{}}
  ↪ }\NormalTok{ unpaid }\OperatorTok{=}\NormalTok{
  ↪ billingDAO}\OperatorTok{.}\FunctionTok{findUnpaidBills}\OperatorTok{()};}
\ControlFlowTok{return}\NormalTok{
  ↪ unpaid}\OperatorTok{.}\FunctionTok{stream}\OperatorTok{()}\
    \OperatorTok{.}\FunctionTok{filter}\OperatorTok{({}\NormalTok{bill
      ↪ }\OperatorTok{-}\textgreater{}}\NormalTok{
      ↪ bill}\OperatorTok{.}\FunctionTok{getDueDate}\OperatorTok{()}\
      ↪ \OperatorTok{!=}\KeywordTok{null}
        \OperatorTok{\&\&}\NormalTok{ bill}\OperatorTok{.}\FunctionTok{getDueD
          ↪ ate}\OperatorTok{()}\FunctionTok{isBefore}\OperatorTok{({}\NormalTok{
          ↪ Tok{LocalDate}\OperatorTok{.}\FunctionTok{now}\OperatorTok{()})}}
        \OperatorTok{.}\FunctionTok{collect}\OperatorTok{({}\NormalTok{Collectors}\Ope
          ↪ ratorTok{.}\FunctionTok{toList}\OperatorTok{()});}
\OperatorTok{\};}

```

Equivalent without a lambda:

```

\BuiltInTok{Callable}\OperatorTok{\textless{}}\BuiltInTok{List}\OperatorTok{\textless{}}\
↪ NormalTok{Bill}\OperatorTok{\textgreater{}}\textgreater{}}\NormalTok{ overdueScanTask
↪ }\OperatorTok{=}\KeywordTok{new}
↪ \BuiltInTok{Callable}\OperatorTok{\textless{}}\BuiltInTok{List}\OperatorTok{\textless{}}
↪ {\}}\NormalTok{Bill}\OperatorTok{\textgreater{}}\textgreater{}}()}\OperatorTok{\{
  \AttributeTok{@Override}
  \KeywordTok{public} \BuiltInTok{List}\OperatorTok{\textless{}}\NormalTok{Bill}\OperatorTok{\textgreater{}}
  ↪ orTok{\textgreater{}} \FunctionTok{call}\OperatorTok{()}\KeywordTok{throws}
  ↪ \BuiltInTok{Exception} \OperatorTok{\{
    \BuiltInTok{List}\OperatorTok{\textless{}}\NormalTok{Bill}\OperatorTok{\textgreater{}}
    ↪ er{}\NormalTok{ unpaid }\OperatorTok{=}\NormalTok{
    ↪ billingDAO}\OperatorTok{.}\FunctionTok{findUnpaidBills}\OperatorTok{()};}
  \BuiltInTok{List}\OperatorTok{\textless{}}\NormalTok{Bill}\OperatorTok{\textgreater{}}
  ↪ er{}\NormalTok{ result }\OperatorTok{=}\KeywordTok{new}
  ↪ \BuiltInTok{ArrayList}\OperatorTok{\textless{}}\textgreater{}}();}
\ControlFlowTok{for} \OperatorTok{({}\NormalTok{Bill bill
  ↪ }\OperatorTok{:}\NormalTok{ unpaid}\OperatorTok{)} }\OperatorTok{\{
    \ControlFlowTok{if} \OperatorTok{({}\NormalTok{bill}\OperatorTok{.}\FunctionTok{
      ↪ {getDueDate}\OperatorTok{()}\OperatorTok{!=}\KeywordTok{null}
        \OperatorTok{\&\&}\NormalTok{ bill}\OperatorTok{.}\FunctionTok{getDueD
          ↪ ate}\OperatorTok{()}\FunctionTok{isBefore}\OperatorTok{({}\NormalTok{
          ↪ Tok{LocalDate}\OperatorTok{.}\FunctionTok{now}\OperatorTok{()})}}
          ↪ \OperatorTok{\{

```

```

\NormalTok{
  ↪ result}\OperatorTok{.}\FunctionTok{add}\OperatorTok{(\}\NormalTok{bill}\OperatorTok{)});}
    \OperatorTok{\}}
    \OperatorTok{\}}
    \ControlFlowTok{return}\NormalTok{ result}\OperatorTok{;}
  \OperatorTok{\}}
\OperatorTok{\};}

```

1.7.4 Scope and effectively final variables

Lambdas in the repository capture method-local values such as `maxPrice`, `keyword`, `workerId`, and `lastAuditId`. Java requires captured local variables to be `final` or `effectively final`. `workerId` is declared `final` before the worker lambda; values used as stream criteria are not reassigned.

A lambda does not automatically improve performance. Stream/lambda code can improve readability, but it can allocate pipeline objects and may be less efficient than a simple loop for tiny hot paths. The repository uses sequential streams; no `parallelStream()` was found.

1.7.5 Lambda interview chain

- What is a lambda? A concise implementation of a functional interface.
- Why a functional interface? The compiler needs one unambiguous abstract method target.
- Lambda versus anonymous class? Lambda has less syntax and does not create a separate this scope in the same way; anonymous classes can declare more members and extend classes.
- Can a lambda access local variables? Yes, if they are `final` or `effectively final`.
- Does a lambda improve performance? Not inherently; it primarily improves expression of behavior.

1.8 7. Stream API

1.8.1 Verified operators

Operator	Found?	Project use
<code>stream()</code>	Yes	plans, bills, subscriptions, complaints, logs, usage
<code>parallelStream()</code>	No	Not implemented/found
<code>filter()</code>	Yes	plan criteria, overdue bills, statuses
<code>map()</code>	Yes	customer/city and audit processing
<code>flatMap()</code>	Not found	Not implemented/found
<code>distinct()</code>	Not found	Not implemented/found
<code>sorted()</code>	Yes	plans and audit logs
<code>limit()</code>	Yes	top 10 customers; first 20 usage records displayed
<code>skip()</code>	Not found	Not implemented/found
<code>peek()</code>	Not found	Not implemented/found
<code>collect()</code>	Yes	lists, grouping, summaries
<code>toList()</code>	Yes	controller filtering
<code>reduce()</code>	Not found	Not implemented/found
<code>count()</code>	Yes	complaint status counts
<code>anyMatch()</code>	Yes	duplicate active subscription check
<code>allMatch()</code>	Not found	Not implemented/found
<code>noneMatch()</code>	Not found	Not implemented/found
<code>findFirst()</code>	Not found in the verified search	Not implemented/found

Operator	Found?	Project use
findAny()	Not found	Not implemented/found
forEach()	Yes	console display and map iteration
mapToInt()	Yes	complaint totals
mapToDouble()	Yes	report totals/statistics
groupingBy()	Yes	revenue and customer distributions
summingDouble()	Yes	revenue grouping
summarizingDouble()	Yes	revenue statistics

1.8.2 Representative pipeline: plan filtering

```
\NormalTok{\textless{}\TelecomPlan\textgreater{}}
\NormalTok{{-}\textgreater{} stream()}
\NormalTok{{-}\textgreater{} filter(Predicate\textless{}\TelecomPlan\textgreater{}}
\NormalTok{{-}\textgreater{} collect(Collectors.toList())}
\NormalTok{{-}\textgreater{} List\textless{}\TelecomPlan\textgreater{}}
```

The stream does not change the original list. Intermediate operations such as `filter` are lazy; the terminal `collect` causes traversal.

1.8.3 Representative pipeline: top customers

`ReportServiceImpl.getHighestConsumingCustomers` groups all bills by customer ID to calculate totals, then sorts customers by their total and limits the result to ten. The implementation is technically a stream-based ranking, but its name is misleading because it uses bill totals rather than usage quantity.

1.8.4 Representative pipeline: audit activity monitor

```
\NormalTok{all audit logs}
\NormalTok{{-}\textgreater{} stream()}
\NormalTok{{-}\textgreater{} filter(log.getAuditId() \textgreater{} lastAuditId)}
\NormalTok{{-}\textgreater{} sorted(Comparator.comparingInt(AuditLog::getAuditId))}
\NormalTok{{-}\textgreater{} collect(toList())}
```

This produces only new logs in ascending audit-ID order.

1.8.5 Stream versus collection

A collection stores data; a stream describes a computation over data. In this project, DAO methods return collections, and services/controllers use streams to filter, sort, group, and aggregate them.

1.8.6 Stream versus loop

Streams make transformation intent concise and composable. Loops are often easier to step through in a debugger and can express early exits/mutable state more directly. The project uses streams for read-oriented transformations and loops for I/O, lifecycle control, and batch processing.

1.8.7 Sequential versus parallel stream

The project uses sequential `stream()` only. A parallel stream is **not implemented/found in this project**. Introducing one would require checking thread safety, database access behavior, ordering requirements, and workload size; it would not automatically improve these console/reporting operations.

1.9 8. Multithreading and Concurrency

1.9.1 Overall status

Multithreading is genuinely implemented. It is not limited to a comment or a class name.

1.9.2 PaymentNotificationService: producer-consumer

- Queue: bounded `LinkedBlockingQueue<Notification>(100)`
- Consumers: three worker tasks in `newFixedThreadPool(3)`
- Producer: `sendNotification(...)`
- Work: persist notification through `AuditAndNotificationDAO`
- Visibility flag: `volatile boolean running`
- Shutdown: set flag, call `shutdown`, await up to ten seconds, then `shutdownNow`

Why used: payment processing should not wait for notification persistence/printing.

If sequential: the payment flow could be delayed by notification database work.

Problems: queue overflow drops notifications; shutdown does not explicitly drain queued messages; multiple workers compete for DAO/database resources.

1.9.3 UsageProcessor: four-worker executor

- Executor: fixed pool of four
- Work: process partitions/batches of usage records
- Database behavior: batch inserts
- Shared state: processed counter protected by `synchronized (lock)`

Why used: independent usage batches can be processed concurrently.

Risk: direct calls with a zero or negative batch size can break partitioning logic; shared counters and database limits must be managed carefully.

1.9.4 AccountMonitor: Callable and Future

- Executor: fixed pool of two
- Tasks: overdue scan and overdue marking/suspension-related work
- Result handling: `Future.get` with timeouts

Important race: the scan and update work can observe/change overlapping bills concurrently. A production design would separate read-only analysis from updates or coordinate through transaction isolation/locking.

1.9.5 BillingScheduler: scheduled work

Uses `ScheduledExecutorService.scheduleAtFixedRate` for a billing task. It is available but not automatically started from `MainApplication`; therefore “automatic billing runs continuously” would be an inaccurate project explanation.

1.9.6 ConsoleActivityMonitor: scheduled polling

Uses `newSingleThreadScheduledExecutor` and polls every two seconds. `start`, `toggle`, and `stop` are synchronized; state flags are volatile.

1.9.7 OTPService: concurrent collection

Uses a static `ConcurrentHashMap` for OTP entries. This protects map operations within the JVM, but does not make the complete OTP workflow automatically atomic or distributed.

1.9.8 Required interview answers

What is multithreading?

Multithreading allows multiple independent execution paths to run within one process. In this project it is used for asynchronous notification persistence, batch usage processing, scheduled work, and account monitoring.

Where did you use it?

The clearest example is `PaymentNotificationService`: successful payments enqueue notifications, and three worker threads consume the bounded queue and persist them.

Why use it?

To keep the user-facing payment workflow from waiting for notification work and to process independent batches concurrently.

Is it thread-safe?

Parts are designed for concurrency: `BlockingQueue`, `ConcurrentHashMap`, volatile lifecycle flags, and synchronized counters/lifecycle methods. The full application is not automatically thread-safe because database workflows, queue shutdown, and account-monitor read/update behavior still require stronger coordination.

Can deadlock occur?

No explicit lock cycle is evident in the reviewed scheduler code, but database row locks and inconsistent lock ordering could create database-level blocking/deadlock under concurrent transactions. The project should keep transactions short and lock rows in a consistent order.

Alternative: use a managed executor, message broker, durable notification table/outbox, database transaction plus outbox publishing, or a scheduled job framework in a web deployment.

1.10 9. Collections

1.10.1 Collections found

Collection	Evidence	Why used
List	DAO/service/controller APIs	Ordered sets of plans, bills, customers, usage, complaints
ArrayList	service/controller/report code	Mutable sequential result lists
Map	report/analytics APIs	Keyed totals, city distributions, statistics
HashMap	report/support code	General key-value lookup without ordering requirement
LinkedHashMap	report/support code	Map iteration in insertion order
EnumMap	analytics/support code	Efficient map keyed by enum values
ConcurrentHashMap	OTPService	Concurrent OTP storage
BlockingQueue	PaymentNotificationService	Thread-safe producer-consumer queue
Collections.emptyList/emptyMap	Map service error/empty results	Immutable empty result values
LinkedList	notification queue implementation (LinkedListBlockingQueue internally is separate)	Any direct application use should be distinguished from queue internals; no main business LinkedList use was identified
TreeMap	Not found	Not implemented/found
TreeSet	Not found	Not implemented/found
Deque	Not found	Not implemented/found
Stack	Not found	Not implemented/found

1.10.2 ArrayList versus LinkedList

The project mainly needs iteration and indexed/domain result handling, so `ArrayList` is a reasonable default. `LinkedList` would only be preferable for frequent insertions/removals in the middle when node references are available; random access is slower. For the notification workflow, a blocking queue is more appropriate than either collection because it supplies synchronization and blocking operations.

1.10.3 HashMap versus TreeMap

The reports need grouping and lookup, not sorted keys, so `HashMap` is appropriate. `TreeMap` would be useful if report output required sorted keys, at the cost of $O(\log n)$ lookup/insertion instead of average $O(1)$ hash lookup.

1.10.4 Complexity points

- `ArrayList.get(index)`: $O(1)$ average.

- ArrayList append: amortized $O(1)$.
- HashMap get/put: average $O(1)$, worst-case depends on collisions/treeification.
- TreeMap get/put: $O(\log n)$.
- Stream filtering/sorting: filtering is generally $O(n)$; sorting is generally $O(n \log n)$.
- BlockingQueue.offer/poll: depends on implementation and contention; the key property is coordination, not asymptotic lookup.

1.11 10. Exception Handling

1.11.1 Exception hierarchy

```
\NormalTok{Exception}
\NormalTok{{-}\textgreater{} TelecomException}
\NormalTok{{-}\textgreater{} ValidationException}
\NormalTok{{-}\textgreater{} AuthenticationException}
```

TelecomException is checked because it extends Exception. Service contracts declare it where business failures are expected.

1.11.2 Actual strategy

- DAOs throw SQLException.
- Services translate database/business failures into TelecomException, ValidationException, or AuthenticationException.
- Controllers catch service exceptions and print user-facing messages.
- JDBC/file resources use try-with-resources.
- Transactional services explicitly rollback on failure.

1.11.3 Good examples

- AuthenticationServiceImpl converts login/database errors into authentication-domain errors.
- PaymentServiceImpl rolls back when payment, bill update, or audit work fails.
- DAO connections/statements/result sets use try-with-resources.
- AccountMonitor handles interruption by restoring the interrupt status where applicable.

1.11.4 Weaknesses

- Several service methods return empty collections, false, 0, or null after database failures, which can hide operational problems.
- UsageServiceImpl.recordUsage returns null on SQL failure.
- Controllers often catch broad Exception.
- Audit/notification failures are sometimes ignored so the primary action can continue.
- BillingScheduler suppresses individual billing exceptions, reducing diagnosability.

1.11.5 Interview answer

"I use checked domain exceptions at the service boundary to separate validation/authentication failures from low-level SQLException. The DAO owns JDBC errors, while the service translates them. The main improvement I would make is to stop converting failures into ambiguous empty results and use consistent error objects/logging, while preserving rollback and interruption semantics."

1.12 11. Database and SQL

1.12.1 Database type

The schema is relational and designed for MySQL-compatible SQL. Runtime selection is:

1. MySQL configured in db.properties.
2. If connection/driver access fails, a file-backed H2 URL is used: jdbc:h2:./data/tcsms_db;DB_CLOSE_DELAY=1;MODE=MySQL.

1.12.2 Tables

src/main/resources/schema.sql defines these 13 tables:

1. customers
2. administrators
3. telecom_plans
4. sim_cards
5. mobile_subscriptions
6. subscription_history
7. usage_records
8. bills
9. payments
10. complaints
11. login_history
12. audit_logs
13. notifications

1.12.3 Key relationships

- mobile_subscriptions.customer_id -> customers.customer_id
- mobile_subscriptions.plan_id -> telecom_plans.plan_id
- mobile_subscriptions.sim_id -> sim_cards.sim_id
- subscription_history.subscription_id -> mobile_subscriptions.subscription_id
- subscription_history.old_plan_id/new_plan_id -> telecom_plans.plan_id
- usage_records.subscription_id -> mobile_subscriptions.subscription_id
- bills.subscription_id -> mobile_subscriptions.subscription_id
- payments.bill_id -> bills.bill_id
- payments.customer_id -> customers.customer_id
- complaints.customer_id -> customers.customer_id
- complaints.subscription_id -> mobile_subscriptions.subscription_id
- notifications.customer_id -> customers.customer_id

1.12.4 Constraints and indexes

- Customer email, username, mobile number, and customer number are unique.
- Plan code, SIM number, IMSI, subscription number, subscription mobile number, bill number, transaction reference, and complaint number are unique.
- mobile_subscriptions.sim_id is unique, preventing assignment of one SIM to multiple subscriptions.
- uq_bill_subscription_month prevents duplicate bills for one subscription and billing month.
- Foreign keys use RESTRICT, CASCADE, and SET NULL according to lifecycle needs.
- Indexes exist for customer email/mobile, subscription customer, usage subscription, bill subscription/status, and complaint status.

1.12.5 JDBC flow

```
\NormalTok{Service implementation}
\NormalTok{{-}\textgreater{} DAO interface}
\NormalTok{{-}\textgreater{} DAO implementation}
\NormalTok{{-}\textgreater{} DBConnection.getConnection()}
\NormalTok{{-}\textgreater{} DriverManager connection}
\NormalTok{{-}\textgreater{} PreparedStatement/Statement}
\NormalTok{{-}\textgreater{} ResultSet or update count}
\NormalTok{{-}\textgreater{} model object / service result}
```

1.12.6 Prepared statements

Most user-controlled values are bound with PreparedStatement, such as identifiers, usernames, email values, statuses, payment data, and complaint filters. This reduces SQL injection risk.

Raw Statement is used for constant queries in list and analytics methods such as Customer-DAOImpl.findAll, PlanDAOImpl.findAll, BillingDAOImpl.findAll, PaymentDAOImpl.findAll, UsageDAOImpl.findAll, complaint analytics, and audit-log listing. These are not direct injection points while the SQL remains constant, but the project README's broad "100% PreparedStatement"

claim is not literally accurate.

1.12.7 Transactions

Payment transaction: `PaymentServiceImpl.processPayment` uses one connection, disables auto-commit, locks the bill with `FOR UPDATE`, inserts the payment, marks the bill paid, writes an audit log, and commits or rolls back.

Plan-change transaction: `SubscriptionDAOImpl.changePlan` disables auto-commit, reads the old plan, updates the subscription, inserts subscription history, and commits or rolls back.

Transaction gap: `SubscriptionServiceImpl.subscribeToPlan` saves the subscription and generates its initial bill through separate service/database operations. If billing fails, the subscription remains. Plan changes and plan-change billing are similarly separated.

1.12.8 Connection handling

`try-with-resources` closes connections, statements, and result sets. There is no connection pool. The singleton is a factory/selection point, not a shared connection pool.

1.12.9 Currency

The schema and Java services use `DOUBLE/double` for money. This can introduce binary floating-point rounding issues. A production implementation should use `DECIMAL` in SQL and `BigDecimal` in Java.

1.13 12. Design Patterns

1.13.1 DAO / Repository-style persistence

Used: DAO interfaces plus JDBC implementations.

Where: `CustomerDAO/CustomerDAOImpl`, `BillingDAO/BillingDAOImpl`, and the other DAO pairs.

Why: isolates SQL and row mapping from service business logic.

Limitation: This is explicitly DAO-oriented rather than a JPA repository; no JPA/Hibernate is present.

1.13.2 Service Layer

Used: service contracts and `service.impl` business implementations.

Why: places validation, business rules, transaction orchestration, and use-case logic between controllers and DAOs.

1.13.3 Factory

Used: `DAOFactory`, `ServiceFactory`, and `PaymentStrategyFactory`.

Why: centralizes selection/creation of implementations.

Limitation: direct construction elsewhere reduces the benefit and creates tight coupling.

1.13.4 Singleton

Used: `DBConnection` with private constructor, static instance, and synchronized `getInstance()`.

Advantages: one process-wide configuration/selection manager.

Disadvantages: global state, difficult tests, hidden dependency, not a connection pool, and process-local behavior.

1.13.5 Strategy

Used: `PaymentStrategy` with UPI/card/net-banking implementations.

Why: isolates payment-channel behavior and supports polymorphic processing.

Limitation: `PaymentStrategyFactory` has a switch, so adding a mode still requires factory modification.

1.13.6 Producer-consumer

Used: PaymentNotificationService.

Why: producers enqueue notifications and consumers process them asynchronously.

1.13.7 Observer-like polling

Partial/observer-like: ConsoleActivityMonitor polls audit records and displays new entries. It is not a classic subject/observer registration implementation; it is scheduled polling.

1.13.8 DTO

Used: CustomerRegistrationDTO separates registration input from the persisted Customer entity.

1.13.9 MVC

Partial console MVC: controllers handle console UI, services hold business logic, and DAO/model layers handle persistence/domain state. It is not web MVC because there is no HTTP layer.

1.13.10 Dependency Injection

Status: Not implemented/found as framework-based DI. There is manual construction and some factory construction, but no Spring container or constructor injection consistently applied.

1.13.11 Builder, Observer, Repository naming, Adapter, Decorator

No explicit Builder, classic Observer registration, Adapter, or Decorator implementation was found. DAO is repository-like but the source names and contracts are DAO.

1.14 13. SOLID Evaluation

1.14.1 S — Single Responsibility

Status: Partially followed.

Good examples: DAO classes focus on persistence; PasswordUtil focuses on hashing; PaymentStrategy implementations focus on payment-mode validation.

Violation: CustomerController and AdminController combine UI, input validation, authorization, service construction, orchestration, reporting, and presentation.

Improvement: Extract console views/input readers, authorization policy, payment workflow coordinator, and report presentation.

1.14.2 O — Open/Closed

Status: Partially followed.

Good example: A new payment strategy can implement PaymentStrategy.

Violation: PaymentStrategyFactory must be edited with another switch branch; enums and UI parsing also need changes.

Improvement: registry/map-based strategy registration or dependency injection.

1.14.3 L — Liskov Substitution

Status: Generally followed for DAO/service/strategy implementations.

Implementations satisfy their interface contracts in normal paths. The main caution is inconsistent failure conventions such as returning null/empty values from some implementations, which makes substitutability less predictable.

1.14.4 I — Interface Segregation

Status: Mostly followed.

The project has focused contracts such as PaymentDAO, PlanDAO, and UsageService. Some larger service/DAO interfaces expose several related operations, but no clear client-forced implementation method problem was established from the source review.

1.14.5 D — Dependency Inversion

Status: Weak/partial.

The interfaces exist, but controllers/services frequently instantiate concrete implementations directly. ServiceFactory also exists but is bypassed in places.

Improvement: inject service/DAO interfaces through constructors, use factories only at the composition root, and pass test doubles in tests.

1.15 14. REST and API Assessment

1.15.1 Status

REST API: Not implemented/found in this project.

Evidence:

- No Spring Boot or Spring Web dependency
- No Jakarta REST/JAX-RS dependency
- No servlet/web-server dependency
- No @RestController, @GetMapping, @PostMapping, route, socket, or JSON API code found
- CustomerController and AdminController are console controllers, not HTTP controllers

Therefore the project has no actual GET/POST/PUT/PATCH/DELETE endpoints, HTTP status-code mapping, JSON request/response contract, or REST authentication filter.

Authentication is an interactive console workflow in AuthenticationServiceImpl, with CAPTCHA, password verification, failed-attempt tracking, account status, and login history.

A future REST version would need request DTOs, JSON serialization, HTTP status mapping, endpoint controllers, centralized exception handling, authentication middleware, and API-level authorization. These are recommendations, not current project facts.

1.16 15. Code Quality, Security, and Performance Review

1.16.1 Strengths

- BCrypt password hashing in PasswordUtil.
- SecureRandom-based CAPTCHA/OTP generation is used in the security utilities.
- CAPTCHA and failed-login lockout are implemented.
- Most dynamic SQL values use PreparedStatement.
- Payment ownership authorization is checked before payment.
- Payment transaction uses row locking and rollback.
- try-with-resources is used extensively for JDBC and file resources.
- Database foreign keys, unique constraints, and indexes are defined.

1.16.2 Findings

Severity	Problem	Location	Why it matters	Recommended fix
High	OTP is printed to console	CustomerController.handleAdminForgotPassword	Anyone viewing the console output can see the OTP	Never print secrets through a protected channel; never print secrets
High	Demo/default credentials are present	seed.sql, README.md, db.properties	Credentials can be reused outside development	Move secrets to environment/secret storage and remove real-looking defaults

Severity	Problem	Location	Why it matters	Recommended fix
High	HTML invoice values are not clearly escaped	ReportGenerator HTML generation	Malicious customer/billing text could become HTML/script content	HTML-escape all inserted values
Medium	OTP storage is process-local	OTPService	Multiple instances do not share OTP state; restart loses active OTPs	Use durable shared storage with expiry and rate limiting
Medium	Account enumeration signal	AuthenticationServiceImpl	Different username/password lookup behavior can reveal account existence	Use uniform authentication responses and timing
Medium	Money uses double/DOUBLE	models/services/scheduler	Floating-point rounding can corrupt currency calculations	Use BigDecimal and SQL DECIMAL
Medium	Cross-service subscription/billing transaction gap	SubscriptionService	Subscription may exist without initial bill after partial failure	Use one transaction/connection or compensating action
Medium	Notification drops on full queue	PaymentNotificationService	Use notifications can be silently lost	Use durable outbox/retry/backpressure/dedup letter handling
Medium	Concurrent account scan/update	AccountMonitor	Results can race on the same bills/accounts	Coordinate tasks and use database isolation/locking
Medium	Errors become empty/false/null results	several services, especially Usage-ServiceImpl	Callers cannot distinguish no data from database failure	Propagate typed failures and log with correlation context
Low	Constructor creates sample complaints	ComplaintService	Production service construction has hidden data mutation	Move sample data to seed/test setup
Low	Notifications viewed but not necessarily marked read	CustomerController	Read/unread state may not reflect user behavior	Call the notification read/update operation after display
Low	Random business IDs can collide	registration/subscription/billing flows	Random/billing identifiers without retry/unique handling can fail or duplicate	Use database-generated identifiers/UUIDs with collision handling
Low	Billing due date is fixed offset	BillingServiceImpl	Due date may not reflect billing month/business calendar	Derive due date from billing period and policy

Severity	Problem	Location	Why it matters	Recommended fix
Low	Report naming/metric mismatch	ReportServiceImpl	"Highest consuming" is based on bill totals, not usage	Rename or calculate usage-based ranking
Low	No automated tests	src/test/java, pom.xml	Regressions are not detected automatically	Add unit, DAO integration, service, and concurrency tests

1.16.3 Interview framing

A strong answer is not "the project is perfect." Say: "The project has good foundations such as BCrypt, parameterized SQL, constraints, transactions for payment, and try-with-resources. I also identified production gaps: double for money, console OTP leakage, process-local OTP state, inconsistent error handling, queue drops, and a transaction boundary that does not cover subscription plus initial billing."

1.17 16. Interview Question Bank

1.17.1 Level 1 — Basic

Q: What does the project do?

Answer: It manages telecom customers, plans, SIMs, subscriptions, usage, bills, payments, complaints, notifications, and reports through a Java console application backed by JDBC and MySQL/H2.

Project example: MainApplication dispatches to CustomerController and AdminController; services and DAOs handle the workflows.

Follow-up: Which workflows are customer versus administrator workflows?

Q: Which Java version and build tool do you use?

Answer: The Maven configuration targets Java 17. pom.xml defines the compiler and exec plugins, while run.bat also provides a direct javac launch path.

Follow-up: Why is the direct batch build a potential maintenance concern?

Q: Is this a REST API?

Answer: No. It is an interactive console application. No web framework, route annotations, JSON API, or HTTP endpoint was found.

Q: What is JDBC doing here?

Answer: DAO implementations obtain connections from DBConnection, bind SQL parameters, execute queries/updates, map rows to models, and close resources.

1.17.2 Level 2 — Intermediate

Q: Explain the payment flow.

Answer: PaymentServiceImpl.processPayment locks and validates the bill, checks customer ownership and amount, selects a PaymentStrategy, inserts payment, marks the bill paid, writes an audit record, commits, and queues a notification.

Follow-up: Why is FOR UPDATE used?

Q: Why use interfaces for services and DAOs?

Answer: They define business/persistence contracts and support substituting implementations. The project benefits from this abstraction, but direct new calls mean dependency inversion is only partial.

Q: What is the difference between DAO and service?

Answer: DAO handles persistence and SQL. Service handles business rules and use-case orchestration. For example, `PaymentDAOImpl` persists records; `PaymentServiceImpl` authorizes payment and coordinates the transaction.

Q: Why use H2 fallback?

Answer: `DBConnection` attempts MySQL but switches to file-backed H2 when MySQL is unavailable, making the console application easier to run locally. The trade-off is that fallback behavior can hide environment/configuration failures.

Q: Where is the Strategy pattern?

Answer: `PaymentStrategy` is the contract, and `UPI/card/net-banking` classes are interchangeable implementations selected by `PaymentStrategyFactory`.

1.17.3 Level 3 — Advanced

Q: Is the payment transaction fully atomic?

Answer: Payment insertion, bill status update, and audit logging are grouped in `PaymentServiceImpl` on one connection with `commit/rollback`. Subscription creation plus initial billing is not fully atomic because those operations are separate.

Q: Is DBConnection a connection pool?

Answer: No. It is a synchronized singleton manager that chooses a database and creates connections through `DriverManager`. A pool such as `HikariCP` is not implemented.

Q: Is the notification system reliable?

Answer: It is asynchronous but not fully durable. It has a bounded queue and three workers, but `offer()` drops notifications when full and shutdown does not explicitly drain the queue.

Q: What concurrency issue exists in AccountMonitor?

Answer: It submits scan and update tasks concurrently, so both can interact with overlapping unpaid-bill/account state. Database isolation or a coordinated transaction is needed for strong consistency.

Q: How would you improve SOLID compliance?

Answer: Split the large controllers, inject service/DAO interfaces through constructors, centralize strategy registration, and make failure contracts consistent rather than returning null or empty values for database errors.

1.17.4 Level 4 — Tricky/follow-up

Q: The README says 100% PreparedStatement. Is that literally true?

Answer: No. Most dynamic queries use `PreparedStatement`, but several constant list and analytics queries use `Statement`. Those current statements do not interpolate user input, so they are not direct injection points, but the README claim is too broad.

Q: Does switch (choice) { case 1 -> ... } prove a lambda is used?

Answer: No. Arrow switch syntax is modern Java syntax, not a lambda. The project does use real lambdas in streams and executor tasks.

Q: Does ConcurrentHashMap make OTP processing fully thread-safe?

Answer: It makes individual map operations safe for concurrent access. It does not make multi-step check/update/expire workflows atomic, durable, or distributed.

Q: Does a stream improve performance?

Answer: Not automatically. It improves expression of sequential transformations. The project uses sequential streams and no `parallelStream`; performance should be measured before parallelizing.

Q: Is getHighestConsumingCustomers measuring network consumption?

Answer: Based on the implementation, it ranks by bill totals. The name suggests usage, so either the metric should be changed to usage quantities or the method renamed.

1.18 17. Interview Question Chains

1.18.1 OOP chain

1. What is encapsulation?
2. Where are private fields used in the models?
3. What is inheritance?
4. Which classes inherit from TelecomException?
5. Where is polymorphism used?
6. How does PaymentStrategy enable runtime substitution?
7. What would change if all strategies were merged into one class?

1.18.2 Collections chain

1. Why return List from DAO methods?
2. When would ArrayList be better than LinkedList?
3. Why use Map for revenue summaries?
4. When would TreeMap be better?
5. Why is ConcurrentHashMap used for OTP storage?
6. Why is BlockingQueue better than a normal list for notifications?

1.18.3 Lambda/functional-interface chain

1. What is a lambda?
2. What is a functional interface?
3. What is SAM?
4. Where is Predicate<TelecomPlan> used?
5. Why does Callable differ from Runnable?
6. What does effectively final mean?
7. Can these lambdas safely mutate shared state?

1.18.4 Stream chain

1. What is the difference between a collection and stream?
2. Which operations are intermediate and terminal?
3. Why is filter lazy?
4. Where is anyMatch used?
5. What is the difference between map and filter?
6. Why is sorting more expensive than filtering?
7. Why was parallelStream not automatically chosen?

1.18.5 Multithreading chain

1. Where are worker threads created?
2. Why use a fixed thread pool?
3. What does BlockingQueue provide?
4. What happens when the queue is full?
5. How is shutdown handled?
6. What data is shared?
7. Where is synchronization used?
8. Could database operations race?
9. How would you make notifications durable?

1.18.6 SQL/database chain

1. Why use a DAO layer?
2. What does PreparedStatement protect against?
3. Why use FOR UPDATE?
4. What is auto-commit?
5. What is rolled back on payment failure?
6. Why is one connection important for a transaction?
7. Why is H2 fallback useful?
8. Why is double a poor currency type?
9. Which foreign keys use cascade/restrict/set-null and why?

1.18.7 Exception chain

1. Why create TelecomException?
2. Is it checked or unchecked?
3. How are SQL errors translated?
4. Why is returning null on SQL failure dangerous?
5. What is the difference between throw and throws?
6. How does try-with-resources work?
7. Why should interruption restore the interrupt flag?

1.18.8 REST/API chain

1. Is this project a REST API?
2. What evidence supports that answer?
3. What would need to be added for REST?
4. How would console AuthenticationService map to HTTP?
5. Where would centralized exception handling go?

1.18.9 Design-pattern chain

1. Where is DAO used?
2. Why is DBConnection a Singleton?
3. What are Singleton drawbacks?
4. Where is Factory used?
5. Where is Strategy used?
6. Why is the factory switch an Open/Closed limitation?
7. Is the activity monitor a true Observer?

1.18.10 SOLID chain

1. Which class most clearly violates SRP?
2. Are interfaces enough to prove DIP?
3. How would constructor injection help testing?
4. How could payment strategy registration improve OCP?
5. What failure contract issue affects LSP?

1.19 18. Project Explanation for Interviews

1.19.1 30-second answer

"I built a Java 17 console-based Telecom Customer and Subscription Management System. It supports customer/admin login, telecom plans, SIM subscriptions, usage, billing, payments, complaints, notifications, and reports. The application uses a service layer over JDBC DAOs, stores data in MySQL with a file-backed H2 fallback, and demonstrates patterns such as DAO, Factory, Strategy, and Singleton. It also has asynchronous notification workers and scheduled-processing examples."

1.19.2 1-minute answer

"The application models telecom operations from customer registration through subscription, billing, payment, and support. MainApplication starts the database and console. Customer and administrator controllers call service interfaces. Service implementations enforce business rules and use DAO interfaces whose JDBC implementations map relational data into model objects. Payment processing is a strong workflow: it validates bill ownership and amount, locks the bill with FOR UPDATE, selects a payment strategy for UPI/card/net banking, writes payment and audit data in one transaction, and queues a notification asynchronously. The schema has customers, plans, SIMs, subscriptions, usage, bills, payments, complaints, audit logs, notifications, and security history. The main limitations are that this is not a REST service, automated tests are absent, and some production concerns remain around money precision, OTP handling, transaction boundaries, and error contracts."

1.19.3 3-minute answer

“The problem is to manage a telecom customer lifecycle with both customer and administrator operations. Customers can register and authenticate, browse plans, subscribe using an available SIM, change plans, view usage and bills, pay bills, raise complaints, and view notifications. Administrators manage plans, customers, subscriptions, billing, usage processing, complaints, reports, and audit activity.

The runtime starts in `MainApplication`. `DBConnection` is a synchronized singleton that loads configuration, attempts MySQL, falls back to file-backed H2, and initializes schema and seed data. The application then starts `PaymentNotificationService` and presents a console menu. The controllers are the UI/application orchestration layer. They call service interfaces, while service implementations contain validation, business rules, payment authorization, report aggregation, and transaction orchestration. DAO interfaces isolate persistence, and `dao.impl` classes use JDBC with prepared statements, result-set mapping, and try-with-resources.

The schema is normalized around customers, administrators, plans, SIMs, subscriptions, subscription history, usage, bills, payments, complaints, login history, audit logs, and notifications. Foreign keys and unique indexes enforce relationships and business constraints. Payment processing demonstrates several concepts together: one connection, manual transaction control, row locking, ownership verification, a strategy factory, audit logging, and asynchronous notification. The strategy classes make UPI, card, and net banking interchangeable, although they currently simulate gateway processing.

The project also demonstrates Java streams, lambdas, `Predicate`, `Runnable`, `Callable`, `Future`, executors, a bounded blocking queue, a concurrent hash map, synchronization, and scheduled tasks. The main weaknesses I would address before production are splitting the large controllers, using constructor injection, using `BigDecimal/DECIMAL` for money, making subscription plus initial billing atomic, protecting OTPs, adding durable notification retry, standardizing error propagation, and adding automated tests. I would also be precise in an interview: no REST API, JPA/Hibernate, connection pool, or verified automated test suite is implemented.”

1.19.4 Discussing contribution

The repository does not identify which person wrote which files. Do not claim a personal contribution that is not documented. A safe interview formulation is: “The implementation I am presenting includes...” and then describe the specific code you can explain. If you personally implemented a subset, name only that subset.

1.20 19. “Why Did You Use This?” Answers

Question	Evidence-based answer
Why Java?	The repository uses Java 17 and benefits from strong typing, JDBC, enums, exceptions, streams, and concurrency APIs. Do not claim a performance benchmark that is not present.
Why MySQL?	<code>db.properties</code> and the MySQL connector support the relational case-study database.
Why H2?	<code>DBConnection</code> provides a local file-backed fallback when MySQL is unavailable.
Why a DAO?	To isolate SQL and row mapping from business services.
Why a service layer?	To keep validation and business workflows above persistence.
Why interfaces?	To define service/DAO/strategy contracts and allow alternate implementations.
Why Factory?	To centralize construction/selection, especially payment strategies.
Why Strategy?	Payment channels have different behavior behind a common contract.

Question	Evidence-based answer
Why ArrayList/List?	Most results are iterated and returned as ordered collections.
Why HashMap?	Report grouping generally needs key lookup, not sorted keys.
Why BlockingQueue?	It coordinates asynchronous notification producers and consumers.
Why ExecutorService?	It manages worker lifecycle and avoids manually creating unmanaged threads.
Why Callable?	Account-monitor tasks return values and can throw checked exceptions.
Why Lambda?	It expresses predicates and task behavior compactly.
Why streams?	Read-oriented filtering, sorting, grouping, and aggregation are concise and composable.
Why transactions?	Payment and plan-history changes require multiple updates to succeed or fail together.
Why BCrypt?	Passwords should be stored as slow salted hashes rather than plaintext.
Why CAPTCHA/OTP?	The console authentication/recovery workflow includes additional verification.
Why not REST?	It was implemented as a console case study; no web framework or HTTP layer is present.
Why not JPA?	Persistence is explicitly JDBC/DAO-based; JPA/Hibernate is not a dependency.

1.21 20. Final Revision Sheet

Concept	1-line definition	Used?	Where	Why / likely question
Encapsulation	Hide state behind controlled methods	Yes	model classes, services, DBConnection	"How do you protect internal state?"
Inheritance	Derive one class from another	Yes, exceptions	ValidationException, AuthenticationException	"Why use custom exception subclasses?"
Polymorphism	Common contract, different runtime implementation	Yes	payment strategies, DAOs, services	"How does payment mode select behavior?"
Abstract class	Partial implementation base type	No	Not implemented/found	"Why did you use interfaces instead?"
Interface	Contract for behavior	Yes	DAO/service/strategy packages	"Does interface usage guarantee DIP?"
Overloading	Same name, different parameters	Yes	DAO findById, audit logAudit	"Why pass a connection overload?"
Overriding	Implementation replaces interface/parent method	Yes	all *Impl, payment strategies	"How is runtime dispatch used?"

Concept	1-line definition	Used?	Where	Why / likely question
Static	Belongs to class rather than instance	Yes	factories, singleton, utilities	"What are static-state testing risks?"
Final	Prevent reassignment/extension as applicable	Yes	constants/dependencies	"Does final make an object immutable?"
Composition	Object owns collaborators/lifecycle	Yes	notification service queue/pool	"How is worker lifecycle managed?"
Predicate	Function returning boolean	Yes	PlanServiceImpl	"Which functional interfaces did you use?"
Runnable	Task with no result	Yes	notification/usage/billing	"Why Runnable rather than Callable?"
Callable	Task returning a value	Yes	AccountMonitor	"How do you obtain the result?"
Function	One input to one output	No direct declaration	Not implemented/found	"Where could it fit?"
Lambda	Compact functional-interface implementation	Yes	streams/executors	"What is effectively final?"
Stream	Lazy computation pipeline over data	Yes	plan/report/controller code	"What is the terminal operation?"
Parallel stream	Parallel stream pipeline	No	Not implemented/found	"Why not parallelize?"
ArrayList	Resizable indexed list	Yes	result collections	"Why not LinkedList?"
HashMap	Hash-based key/value map	Yes	reports/support code	"When would TreeMap fit?"
ConcurrentHashMap	Concurrent key/value map	Yes	OTPService	"What does it not guarantee?"
BlockingQueue	Thread-safe blocking producer/consumer queue	Yes	notification service	"What happens when full?"
Checked exception	Must be handled/declared	Yes	TelecomException	"How are SQL errors translated?"
PreparedStatement	Parameterized JDBC statement	Yes	most DAO writes/lookups	"How does it reduce injection risk?"
Transaction	Atomic group of database operations	Yes, partial	payment and plan change	"Which workflow is not atomic?"
DAO	Persistence abstraction	Yes	dao and dao.impl	"Where does SQL live?"
Service layer	Business/use-case boundary	Yes	service.impl	"What belongs in service versus DAO?"

Concept	1-line definition	Used?	Where	Why / likely question
Factory	Centralized object selection/creation	Yes	factories	"What is the OCP limitation?"
Singleton	One process-level instance	Yes	DBConnection	"Why is it not a connection pool?"
Strategy	Interchangeable algo-rithm/behavior	Yes	payments	"How do you add a payment mode?"
MVC	UI/model/application separation	Partial	console controllers/services/models	"Why is this not a MVC?"
REST	HTTP resource API style	No	Not implemented/found	"What evidence proves that?"
JPA/Hibernate	ORM persistence framework	No	Not implemented/found	"Why JDBC instead?"
Automated tests	Repeatable executable verification	No verified suite	src/test empty/no test dependency	"How would you improve confidence?"

1.22 21. Final Repository Verification

The final claims in this document were checked against the repository's authoritative implementation surfaces:

- Source inventory: 79 Java files under src/main/java.
- Build/dependencies: pom.xml targets Java 17 and contains H2, MySQL Connector/J, and jBCrypt only.
- Entry point: MainApplication.main initializes the database and starts PaymentNotificationService, but not BillingScheduler.
- Database: schema.sql defines 13 tables and the documented foreign keys/indexes.
- Functional interfaces: direct source use confirms Predicate, Runnable, and Callable; no direct named Function, Consumer, Supplier, BiFunction, BiConsumer, UnaryOperator, or BinaryOperator declaration was found.
- Streams: direct source use confirms sequential streams and the operators listed above; no parallelStream, flatMap, reduce, or distinct usage was found.
- Concurrency: direct source use confirms executors, futures, scheduled executors, blocking queue, concurrent map, volatile state, and synchronized blocks/methods.
- Persistence: DAO implementations use JDBC, mostly prepared statements, explicit transactions in payment/plan-change paths, and try-with-resources.
- REST: no web framework, endpoint annotations, HTTP server, or JSON API was found.
- Tests: no current test source files or test dependency was found; compiled artifacts under bin/target were not treated as source evidence.

This is the safest interview position: explain what is implemented precisely, acknowledge partial/weak areas, and label absent technologies and capabilities instead of implying they exist.